

PrismViz 组件交付说明

PrismViz 是面向甲方的组件化交付包。当前交付包含五个可独立调用的可视化组件、reference/client 两套示例数据、原生 HTML demo、React demo、interaction coordinator、多组件交互 demo，以及一份从数据到调用的完整说明。组件既可以单独接入，也可以通过 coordinator 实现“调用哪几个组件，就启用哪几个组件之间的交互”。

交付内容

PrismViz/	
src/	组件源码、数据建模、数据适配、详情格式化、运行时依赖
instances/	
reference/	reference 数据、原生 demo、React demo、多组件交互 demo
client/	client 数据、原生 demo、React demo、多组件交互 demo
shared/	reference/client demo 共用页面外壳样式
example/	原系统交互示例，仅用于参考 legacy 效果
README.md	唯一主文档
README.pdf	README 的 PDF 版本

包含的五个组件：

- Tree：slice/topic 层级列表入口。由于产业通已经包含了较好的层次树可视化，故本组件仅做了层次展开，以做可视交互效果展示，需与甲方重点联调。
- Chord：slice 之间的关系总览。
- Prism：三维棱镜式多 slice 总览。
- Scroll：单个 slice 内部点边图，支持 time layered 或 free graph，由于甲方数据没有严格一对一的时间层次，所以默认使用的 free graph 布局
- List：节点、边、slice、stream 等点击结果的详情面板，需进行详细的微调，确定展示信息

边界说明：

- src/ 只保留组件自身功能、数据建模、数据适配、详情格式化、运行时依赖和可选 interaction coordinator。
- instances/ 只保留各组件自己的 demo 和对应数据的可视化实例。
- instances/shared/ 只服务 demo 页面外壳，不属于组件 API。
- src/runtime/ 保存 Prism/Scroll legacy 渲染需要的轻量运行时依赖。
- example/ 保留源系统示例，用来查看原系统交互效果，不作为新组件依赖。
- 不包含旧合包数据 prismviz_data.json。
- 单组件 demo 不依赖源系统旧合包脚本。

两套数据

reference 数据

instances/reference/data/ 是原 GeneticPrism/reference 学术数据的组件化版本。它以学术论文为节点、引用关系为边，slice 表示论文主题/研究方向。该数据用于复现原系统中 Prism、Scroll、Chord、List、Tree 的视觉效果，尤其是 Scroll 的 time layered 布局。

reference 数据文件：

```
instances/reference/data/  
  slice_definitions.json  
  slice_hierarchy.json  
  entities.json  
  relations.json  
  display_info.json  
  component_options.json
```

client 数据

instances/client/data/ 是甲方业务形态的示例数据。它以企业/对象为节点，以投资、股权或其他业务关系为边，slice 表示产业环节或业务分类。client 数据用于演示甲方数据接入组件后的效果。

client 数据文件和 reference 保持一致：

```
instances/client/data/  
  slice_definitions.json  
  slice_hierarchy.json  
  entities.json  
  relations.json  
  display_info.json  
  component_options.json
```

两套数据的目标是进入统一模型后走同一批组件入口。差异只发生在最前面的 adapter：reference 主要保留原学术语义；client 会把产业/企业字段整理成组件统一字段，并对同一点对点之间的多条关系做聚合保留。

快速打开 Demo

不要直接双击 HTML。请在仓库根目录启动静态服务：

```
cd PrismViz  
python3 -m http.server 8765
```

reference 原生 demo：

```
http://127.0.0.1:8765/instances/reference/demos/tree.html  
http://127.0.0.1:8765/instances/reference/demos/chord.html  
http://127.0.0.1:8765/instances/reference/demos/prism.html  
http://127.0.0.1:8765/instances/reference/demos/scroll.html  
http://127.0.0.1:8765/instances/reference/demos/list.html
```

client 原生 demo：

```
http://127.0.0.1:8765/instances/client/demos/tree.html  
http://127.0.0.1:8765/instances/client/demos/chord.html  
http://127.0.0.1:8765/instances/client/demos/prism.html  
http://127.0.0.1:8765/instances/client/demos/scroll.html  
http://127.0.0.1:8765/instances/client/demos/list.html
```

reference React demo:

```
http://127.0.0.1:8765/instances/reference/demos-react/tree.html
http://127.0.0.1:8765/instances/reference/demos-react/chord.html
http://127.0.0.1:8765/instances/reference/demos-react/prism.html
http://127.0.0.1:8765/instances/reference/demos-react/scroll.html
http://127.0.0.1:8765/instances/reference/demos-react/list.html
```

client React demo:

```
http://127.0.0.1:8765/instances/client/demos-react/tree.html
http://127.0.0.1:8765/instances/client/demos-react/chord.html
http://127.0.0.1:8765/instances/client/demos-react/prism.html
http://127.0.0.1:8765/instances/client/demos-react/scroll.html
http://127.0.0.1:8765/instances/client/demos-react/list.html
```

多组件交互 demo:

```
http://127.0.0.1:8765/instances/reference/demos/interactions/tree-chord.html
http://127.0.0.1:8765/instances/reference/demos/interactions/tree-prism.html
http://127.0.0.1:8765/instances/reference/demos/interactions/tree-scroll.html
http://127.0.0.1:8765/instances/reference/demos/interactions/chord-prism.html
http://127.0.0.1:8765/instances/reference/demos/interactions/chord-scroll.html
http://127.0.0.1:8765/instances/reference/demos/interactions/prism-scroll.html
http://127.0.0.1:8765/instances/reference/demos/interactions/tree-chord-prism.html
http://127.0.0.1:8765/instances/reference/demos/interactions/tree-chord-scroll.html
http://127.0.0.1:8765/instances/reference/demos/interactions/tree-prism-scroll.html
http://127.0.0.1:8765/instances/reference/demos/interactions/chord-prism-scroll.html
http://127.0.0.1:8765/instances/reference/demos/interactions/tree-chord-prism-scroll.html
```

client 交互 demo 路径与 reference 对称，只需把路径中的 reference 换成 client。两两 demo 6 个、三组件 demo 4 个、四组件 demo 1 个；reference/client 合计 22 个交互页面。

数据到调用的完整链路

推荐把甲方数据整理成 6 个对象或 6 个 JSON 文件：

```
slice_definitions.json  -> slices
slice_hierarchy.json    -> hierarchy
entities.json           -> entities
relations.json          -> relations
display_info.json       -> displayInfo
component_options.json  -> config
```

调用链路：

```
原始 JSON
-> src/adapters/reference.js 或 src/adapters/client.js
-> src/core/model.js 的统一建模
-> model.tree / model.chord / model.prism / model.scrollBySliceId / model.list
```

```
-> renderTree / renderChord / renderPrism / renderScroll / renderList  
-> 或 ReactTree / ReactChord / ReactPrism / ReactScroll / ReactList
```

正式接入时，甲方不需要复制或编写额外的 demo 辅助脚本。当前 demo 页面只是自己 fetch 本地 JSON，然后把数据对象传给 src/adapters 或 src/core。

数据格式

最小输入对象：

```
const input = {  
  slices,  
  hierarchy,  
  entities,  
  relations,  
  displayInfo,  
  config,  
};
```

最小 slice：

```
{  
  "id": "slice_25351",  
  "name": "Fabless设计",  
  "shortName": "Fabless设计",  
  "color": "#f6b35a"  
}
```

最小节点：

```
{  
  "id": "149048320",  
  "label": "北京电科智芯科技有限公司",  
  "primarySliceId": "slice_25351",  
  "sliceWeights": {"slice_25351": 1},  
  "impact": 5000,  
  "importance": 0.0001,  
  "time": "2020",  
  "meta": {"法人": "张鹏", "状态": "存续"}  
}
```

最小关系：

```
{  
  "id": "rel_001",  
  "source": "149048320",  
  "target": "214995632",  
  "type": "investment",  
  "weight": 1,  
  "time": "2020",  
}
```

```
"meta": {"持股比例": "100.0%"}
}
```

display_info.json 用于控制 List Panel 展示字段，建议预处理后只放语义最相关的内容：

```
{
  "entityInfoById": {
    "149048320": {
      "title": "北京电科智芯科技有限公司",
      "subtitle": "110100",
      "metrics": {"注册资本(万元)": 5000},
      "attributes": {"法人": "张鹏", "状态": "存续"}
    }
  },
  "relationInfoById": {
    "rel_001": {
      "title": "投资关系",
      "subtitle": "2020",
      "attributes": {"持股比例": "100.0%"}
    }
  }
}
```

Adapter 入口

推荐统一从 src/adapters/index.js 引入：

```
import {
  normalizeReferenceInput,
  buildReferenceModels,
  referenceDetailRows,
  getReferenceDisplayInfo,
  normalizeClientInput,
  buildClientModels,
  capScrollModelBySliceWeights,
  clientDetailRows,
  getClientDisplayInfo,
} from './src/adapters/index.js';
```

也可以按数据类型直接引入：

```
import {
  buildReferenceModels,
  detailRows as referenceDetailRows,
} from './src/adapters/reference.js';
import {
  buildClientModels,
  detailRows as clientDetailRows,
} from './src/adapters/client.js';
```

reference:

```
const model = buildReferenceModels({
  slices,
  hierarchy,
  entities,
  relations,
  displayInfo,
  config,
});
```

client:

```
const model = buildClientModels({
  slices,
  hierarchy,
  entities,
  relations,
  displayInfo,
  config,
});
```

高级底层入口一般不需要使用。常规接入建议始终走 `reference.js` 或 `client.js`，只有在已经自行完成标准化数据建模时，才直接使用 `core`：

```
import {buildComponentModels} from "./src/core/index.js";

const model = buildComponentModels({
  normalized: true,
  slices,
  hierarchy,
  entities,
  relations,
  displayInfo,
  config,
});
```

原生 HTML 调用

原生调用最小模板：

```
<link rel="stylesheet" href="./src/components/tree/tree.css">
<div id="tree" style="width: 960px; height: 640px;"></div>
<script type="module" src="./demo.js"></script>
```

```
import {buildClientModels} from "./src/adapters/index.js";
import {renderTree} from "./src/components/tree/Tree.js";

const model = buildClientModels(input);
```

```
renderTree("#tree", model.tree, {
  maxLevel: 2,
  showCount: true,
  onSelect: node => console.log(node),
});
```

Scroll、Prism、Chord 这类可视化组件的外层容器必须有明确宽高。instances/*/demos/*.html 是可直接参考的完整原生调用示例。

instances/shared/demo.css 和 instances/shared/react-demo.css 只用于交付包 demo 页面外壳。甲方正式集成时应优先引用 src/components/*/*.css 和对应组件入口。

React 调用

React 工程里引入 wrapper:

```
import {ReactTree} from "./src/components/tree/Tree.react.js";
import {ReactChord} from "./src/components/chord/Chord.react.js";
import {ReactPrism, ReactPrismSliceTags} from "./src/components/prism/Prism.react.js";
import {ReactScroll} from "./src/components/scroll/Scroll.react.js";
import {ReactList} from "./src/components/list/List.react.js";
```

同时引入样式:

```
import "./src/components/tree/tree.css";
import "./src/components/chord/chord.css";
import "./src/components/prism/prism.css";
import "./src/components/scroll/scroll.css";
import "./src/components/list/list.css";
```

React Scroll 最小示例:

```
function ScrollDemo({model, sliceId, onSelect}) {
  return (
    <div style={{height: 640}}>
      <ReactScroll
        data={model.scrollBySliceId[sliceId]}
        options={{
          layoutMode: "free-graph",
          scrollRenderer: "graph",
          withContext: true,
          showStreams: true,
          componentModels: model,
          slices: model.slices,
          colorMap: model.colorMap,
        }}
        onSelect={onSelect}
      />
    </div>
  );
}
```

```
);  
}
```

instances/*/demos-react/*.html 是不依赖构建工具的 React CDN 示例。甲方正式工程可以把这些写法迁移到自己的 React/Vite/webpack 项目中。

组件 API

Tree

调用入口：

```
const treeInstance = renderTree(container, model.tree, options);
```

React：

```
<ReactTree data={model.tree} options={options} onSelect={handleSelect} />
```

常用 options：

- maxLevel：最多显示层级。
- showCount：是否显示数量。
- onSelect：选择节点回调。
- onHover：hover 节点回调。

实例方法：

- update(nextData, nextOptions)
- destroy()

说明：产业通系统已有较成熟的 Topic Tree/产业层级树实现；原系统中也没有深度开发 Tree 组件。因此本包里的 Tree 是为了配合组件展示和后续联动做的轻量列表实现。

Chord

调用入口：

```
const chordInstance = renderChord(container, model.chord, options);
```

React：

```
<ReactChord data={model.chord} options={options} onSelect={handleSelect} />
```

常用 options：

- onSelect：点击 slice 或 ribbon。
- onSliceHover：hover slice。
- onRelationHover：hover slice 间关系。

实例方法：

- highlightSlice(sliceId)

- highlightRelation(sourceSliceId, targetSliceId)
- clearHighlight()
- update(nextData, nextOptions)
- destroy()

Prism

调用入口：

```
const prismInstance = renderPrism(container, model.prism, options);
const tagInstance = renderPrismSliceTags(tagContainer, model.slices, tagOptions);
```

React：

```
<ReactPrism data={model.prism} options={options} />
<ReactPrismSliceTags slices={model.slices} options={tagOptions} onSelect={handleSlice} />
```

常用 options：

- onSelect：点击 Prism 元素。
- activeSliceId：当前 slice。
- colorMap：slice 颜色。
- widthMode：weighted 或 equal。
- faceGraphLayoutMode：面内布局方式。
- autoRotate / rotationSpeed：自动旋转控制。

实例方法：

- update(nextData, nextOptions)
- destroy()

Scroll

调用入口：

```
const scrollInstance = renderScroll(container, model.scrollBySliceId[sliceId], options);
```

React：

```
<ReactScroll data={model.scrollBySliceId[sliceId]} options={options} onSelect={handleSelect} />
```

reference 支持：

- layoutMode: "layered-time": time layered。
- layoutMode: "free-graph": free graph。

client 当前只使用 free-graph。reference/client 的 free graph 使用同一套 Scroll 入口。

常用 options：

- layoutMode：layered-time 或 free-graph。
- scrollRenderer：layered-scroll 或 graph。
- withContext：是否保留上下文关系。
- showStreams：是否展示左右 stream。

- componentModels: 完整模型, 用于详情和跨 slice 查询。
- slices: slice 列表。
- colorMap: slice 颜色。

回调:

- onSelect(item, context)
- onNodeHover(node, context)
- onEdgeHover(edge, context)
- onSegmentSelect(segment, context)
- onStreamSelect(stream, context)

实例方法:

- highlightEntity(entityId)
- highlightRelation(relationOrId)
- highlightContextSlice(sliceId, direction)
- update(nextData, nextOptions)
- destroy()

Scroll 下方的 slice 标签不是 Scroll 内部 DOM, 而是独立的 Prism 标签组件:

```
renderPrismSliceTags("#slice-tags", model.slices, {  
  activeSliceId,  
  colorMap: model.colorMap,  
  onSelect: slice => {},  
});
```

client Scroll 如果需要限制单个 slice 的节点数, 可以使用:

```
const scrollModel = capScrollModelBySliceWeights(model, sliceId, 300);
```

List

调用入口:

```
const listInstance = renderList(container, detailData, options);
```

React:

```
<ReactList data={detailData} options={options} />
```

常用 options:

- mode: json、list、table、auto。
- numberPrecision: 数值精度。
- title: 标题。

用法:

```
import {detailRows} from "./src/adapters/client.js";  
// reference 数据对应从 "./src/adapters/reference.js" 引入同名 detailRows
```

```
renderList("#detail", detailRows(model, item, context), {  
  mode: "json",  
  numberPrecision: 4,  
});
```

List 可以直接接收 Tree、Chord、Prism、Scroll 点击回调中的数据，也可以接收 detailRows(model, item, context) 的语义整理结果。对于 client 中同一点的多条关系，聚合边详情会显示 title: "多重关系"，并在 relations 字段下逐条展开原始 relation。List 是详情展示接收端，不进入组件间 coordinator 的联动规划。

组件间交互调用

组件间联动由 src/interaction/index.js 中的 createInteractionCoordinator() 提供。Tree、Chord、Prism、Scroll 可以任意组合注册；List 只做详情展示接收端，不参与 coordinator 联动矩阵。

coordinator 最小入口：

```
import {createInteractionCoordinator} from "./src/interaction/index.js";  
  
const coordinator = createInteractionCoordinator({  
  model,  
  initialSliceId,  
  getScrollData: sliceId => model.scrollBySliceId[sliceId],  
  onScrollSliceChange: (sliceId, info) => {  
    // 在包含 Scroll 的组合页中，切换中间主视图并重渲染 Scroll。  
  },  
  onViewChange: view => {  
    // view 为 "prism" 或 "scroll"。  
  },  
  onStateChange: state => {  
    // 可用于调试或同步页面状态栏。  
  },  
});
```

原生多组件调用示例：

```
import {buildClientModels, detailRows} from "./src/adapters/client.js";  
import {createInteractionCoordinator} from "./src/interaction/index.js";  
import {renderTree} from "./src/components/tree/Tree.js";  
import {renderChord} from "./src/components/chord/Chord.js";  
import {renderPrism, renderPrismSliceTags} from "./src/components/prism/Prism.js";  
import {renderScroll} from "./src/components/scroll/Scroll.js";  
import {renderList} from "./src/components/list/List.js";  
  
const model = buildClientModels(input);  
let activeSliceId = model.slices[0].id;  
let scrollInstance = null;  
  
const coordinator = createInteractionCoordinator({  
  model,  
  initialSliceId: activeSliceId,  
  getScrollData: sliceId => model.scrollBySliceId[sliceId],
```

```

onScrollSliceChange: (sliceId, {scrollData}) => {
  activeSliceId = sliceId;
  scrollInstance?.destroy?.();
  scrollInstance = renderScroll("#main", scrollData, coordinator.optionsFor("scroll", {
    layoutMode: "free-graph",
    componentModels: model,
    slices: model.slices,
    colorMap: model.colorMap,
    onSelect: (item, context) => {
      renderList("#list", detailRows(model, item, context));
    },
  }));
  coordinator.register("scroll", scrollInstance);
},
});

coordinator.register("tree", renderTree("#tree", model.tree, coordinator.optionsFor("tree")));
coordinator.register("chord", renderChord("#chord", model.chord, coordinator.optionsFor("chord")));
coordinator.register("prism", renderPrism("#main", model.prism, coordinator.optionsFor("prism")));
renderPrismSliceTags("#tags", model.slices, coordinator.optionsFor("prism", {
  activeSliceId,
  colorMap: model.colorMap,
}));

```

React 多组件调用示例:

```

import {useMemo, useState} from "react";
import {buildClientModels, detailRows} from "../src/adapters/client.js";
import {createInteractionCoordinator} from "../src/interaction/index.js";
import {ReactTree} from "../src/components/tree/Tree.react.js";
import {ReactChord} from "../src/components/chord/Chord.react.js";
import {ReactPrism, ReactPrismSliceTags} from "../src/components/prism/Prism.react.js";
import {ReactScroll} from "../src/components/scroll/Scroll.react.js";
import {ReactList} from "../src/components/list/List.react.js";

function InteractionView({input}) {
  const model = useMemo(() => buildClientModels(input), [input]);
  const [activeSliceId, setActiveSliceId] = useState(model.slices[0].id);
  const [view, setView] = useState("prism");
  const [detailData, setDetailData] = useState([]);

  const coordinator = useMemo(() => createInteractionCoordinator({
    model,
    initialSliceId: activeSliceId,
    getScrollData: sliceId => model.scrollBySliceId[sliceId],
    onScrollSliceChange: sliceId => {
      setActiveSliceId(sliceId);
      setView("scroll");
    },
    onViewChange: setView,
  }), [model]);

  return (

```

```

<>
<ReactTree
  data={model.tree}
  options={coordinator.optionsFor("tree")}
  onReady={instance => coordinator.register("tree", instance)}
/>
<ReactChord
  data={model.chord}
  options={coordinator.optionsFor("chord")}
  onReady={instance => coordinator.register("chord", instance)}
/>
{view === "prism" ? (
  <ReactPrism
    data={model.prism}
    options={coordinator.optionsFor("prism")}
    onReady={instance => coordinator.register("prism", instance)}
  />
) : (
  <ReactScroll
    data={model.scrollBySliceId[activeSliceId]}
    options={coordinator.optionsFor("scroll", {
      layoutMode: "free-graph",
      componentModels: model,
      slices: model.slices,
      colorMap: model.colorMap,
      onSelect: (item, context) => setDetailData(detailRows(model, item, context)),
    })}
    onReady={instance => coordinator.register("scroll", instance)}
  />
)}
<ReactPrismSliceTags
  slices={model.slices}
  options={{activeSliceId, colorMap: model.colorMap}}
  onSelect={slice => {
    if (view === "scroll" && slice.id === activeSliceId) coordinator.setMainView("prism");
    else coordinator.selectSlice(slice, {source: "react-tags"});
  }}
/>
<ReactList data={detailData} />
</>
);
}

```

交互规则：

- Tree 点击任意层级节点时，会向上归一到最近的 slice，再同步 Chord、Prism、Scroll。
- Chord hover 只保留 Chord 内部 hover；点击 arc 选择 slice，点击 ribbon 选择关系。
- Prism 自动旋转或拖动产生的 active face 变化只做轻量同步；点击 slice tag 才视为明确选择。
- Prism + Scroll 同时存在时，默认显示 Prism；明确选择 slice 后切到 Scroll；点击当前 Scroll slice tag 返回 Prism。
- Scroll axis segment 或 stream hover/select 会同步上下文 slice/relation；Scroll node/edge 点击只更新 List。
- Tree + Chord + Prism + Scroll 的完整页面见 instances/*/demos/interactions/tree-chord-prism-scroll.html。

原系统可参考 <example/index.html> 及同目录源码，但当前 demo 运行时不依赖 <example/>。

接入建议

最稳妥的联调方式：

1. 将整个 PrismViz/ 放进甲方 React 工程根目录。
2. 先用 instances/client/data/ 的格式替换为甲方真实数据。
3. 调用 buildClientModels() 生成统一模型。
4. 先接入单个组件，例如 Scroll 或 Prism。
5. 确认单组件数据格式和视觉效果稳定后，再用 createInteractionCoordinator() 注册需要联动的 Tree/Chord/Prism/Scroll。
6. 后续如果需要做成内部 npm 包，可以在接入稳定后再封装。

验收检查

当前交付包应满足：

- src/adapters/reference.js 和 src/adapters/client.js 对称命名。
- reference/client demo 共用 instances/shared/ 下的 demo 页面样式。
- Prism/Scroll demo 从 src/runtime/ 读取运行时依赖，不从 example/ 读取。
- instances/ 下没有额外的 demo 辅助脚本目录。
- demo 页面直接读取本地 JSON，不要求甲方复制额外辅助文件。
- docs/ 不再恢复，主说明集中在 README.md 和 README.pdf。
- example/ 保留为源系统参考。
- src/interaction/ 提供可选 coordinator；调用几个组件，就注册几个组件。
- src/components/scroll/legacyLayeredScroll.js 保留，因为 reference Scroll 的 time layered 仍依赖它。